

SMART INDIA HACKATHON 2026

Problem Statement ID: SIH26156 | Category: Software | Theme: Miscellaneous

ULPF

Universal Log Pre-processing Framework

“Different Logs. One Standard. Trusted Everywhere.”

TECHNICAL REQUIREMENTS DOCUMENT

Architecture, component design, data model, APIs and deployment

Team & Role Allocation

Member	Role	Primary Ownership in This Document
Divyansh Rajat	Team Lead / System Architect	Overall architecture, scope, requirement baseline, traceability
Ashish Bhardwaj	Log Processing Engineer	Ingestion, format detection, template mining, parser factory
Sakshi	Normalization & Schema Engineer	Unified event model, OCSF/ECS mapping, schema registry, traceability
Parampreet Kaur	Backend & Integration Engineer	APIs, persistence, deployment, downstream connectors
Aryan Jeet	Frontend Engineer	Review console, dashboards, visualization requirements
Saksham	Testing, Documentation & Demo	Test strategy, benchmark methodology, acceptance evidence

Reference Documents

Ref.	Document	Source
R1	SIH 2026 Problem Statement SIH26156 — Universal Log Pre-processing Framework	Smart India Hackathon 2026 portal
R2	Drain3 — online log template mining library	logPai / IBM Drain3, open source
R3	OCSF — Open Cybersecurity Schema Framework	ocsf.io schema specification
R4	ECS — Elastic Common Schema field reference	Elastic documentation
R5	Splunk Common Information Model (CIM) documentation	Splunk Docs
R6	IBM QRadar DSM and DSM Editor documentation	IBM Documentation
R7	RFC 5424 — The Syslog Protocol	IETF
R8	ArcSight Common Event Format (CEF) and IBM LEEF specifications	Vendor specifications

1. Introduction

1.1 Purpose

This Technical Requirements Document specifies the architecture, component design, data structures, algorithms, interfaces, deployment model and engineering constraints for ULPF, the Universal Log Pre-processing Framework. Where the SRS states what the system shall do, this document states how it is built and why those choices were made.

It is written for the implementation team and for any technical evaluator who wants to assess whether the design is buildable, defensible and honest about its limits.

1.2 Design Philosophy

Five engineering commitments govern every decision in this document.

#	Commitment	Consequence for the design
1	Deterministic first, probabilistic second	A statistical model is invoked only where a deterministic rule cannot decide. Rules always take precedence, and inference never overrides a matched rule.
2	Fail safe, never fail silent	Every failure path produces an artefact — a dead letter, a low-confidence flag, a review item — rather than a dropped event or a silently wrong value.
3	Preserve before you transform	The raw event is durably stored and digested before any parsing begins. Transformation is always derivative, never destructive.
4	Version everything that decides	Schemas, mappings and templates are immutable and versioned, so historical output remains explicable.
5	Assume no network	Every runtime dependency is resolved at build time. The processing path makes no outbound call, ever.

1.3 Scope Boundaries

The system's technical boundary begins at the ingestion socket or file handle and ends at the delivery sink. It does not include collection agents, detection logic, correlation, alerting, case management, or long-term archival. Enrichment (threat intelligence, geo-IP, asset context) is deliberately excluded and left to downstream consumers, so that ULPF's output remains a faithful representation of what the device reported rather than an interpretation of it.

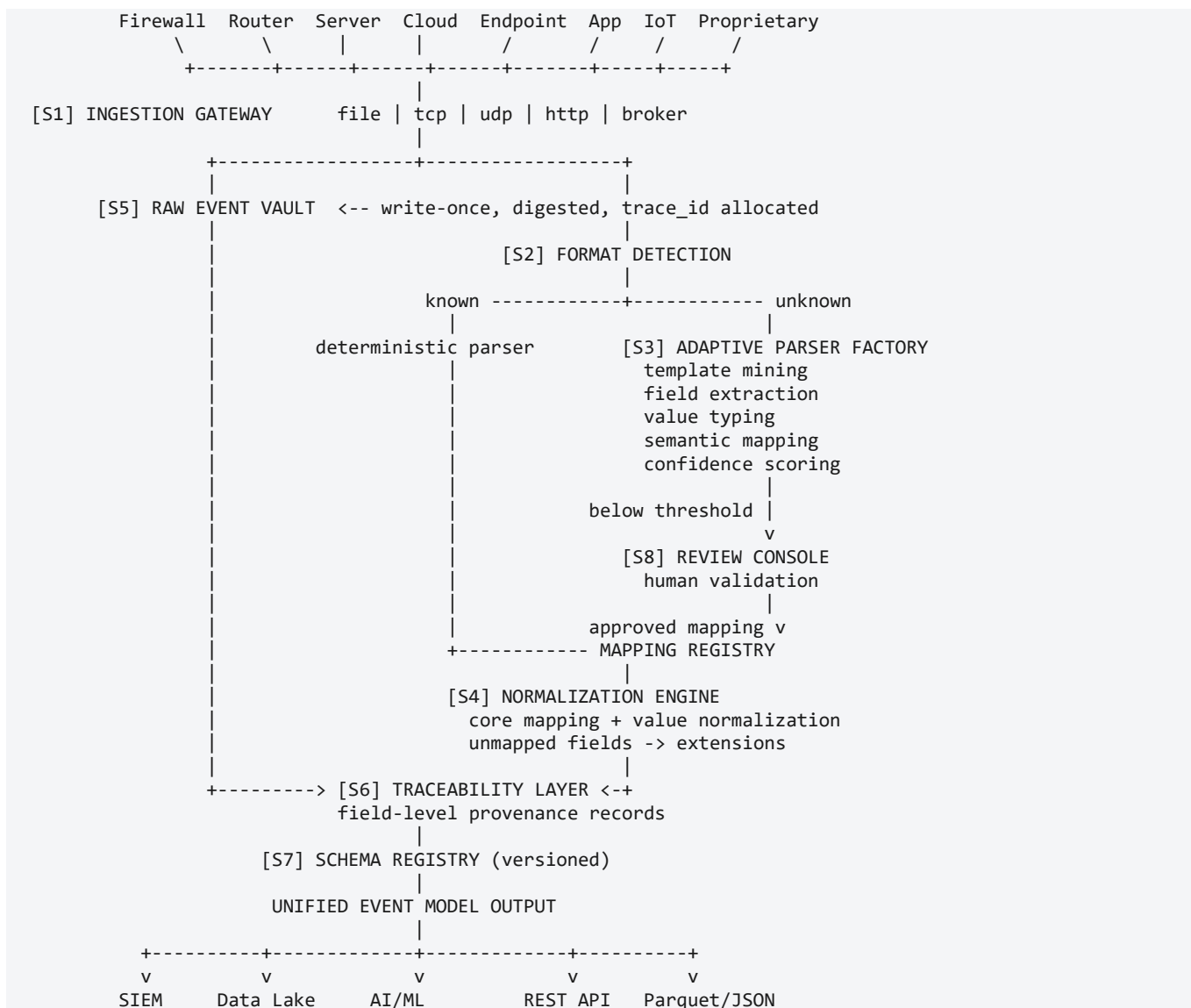
2. System Architecture

2.1 Architectural Style

ULPF is a staged, pipeline-oriented system with a shared persistence tier. Each stage is a separate module with a defined input contract, output contract and failure contract. Stages communicate through an internal queue abstraction so that the deployment can run them in a single process for the prototype or as separate scaled workers in production without changing stage code.

Two design properties follow from this style and matter more than the style itself. First, the Raw Event Vault write happens at stage boundary zero, before anything else, which is what makes the lossless guarantee structural rather than aspirational. Second, the fast path and the adaptive path share every stage except discovery and mapping, so a source that has been onboarded costs no more to process than a source with a hand-written parser.

2.2 Logical Architecture



2.3 Component Responsibilities

Component	Responsibility	Must not
S1 Ingestion Gateway	Accept bytes, bind to a source identity, allocate trace_id, hand off to the vault	Parse, interpret or modify payload content
S2 Format Detection	Classify structural family with a confidence value; route known vs unknown	Reject an event because it is unrecognised

Component	Responsibility	Must not
S3 Adaptive Parser Factory	Discover templates, extract fields, infer types, propose mappings, score confidence	Apply a mapping below the auto-accept threshold
S4 Normalization Engine	Map to core schema, normalize values, place remainder in extensions	Discard any field for any reason
S5 Raw Event Vault	Store the original bytes immutably with a digest and reception metadata	Permit modification or in-place deletion outside retention policy
S6 Traceability Layer	Maintain trace bindings and per-field derivation records; serve lookup	Infer provenance that was not recorded at processing time
S7 Schema Registry	Serve immutable versioned schemas, mapping profiles and extension namespaces	Mutate a published version
S8 Review Console & API	Present proposals with evidence, capture decisions, expose query and provenance APIs	Auto-approve on the user's behalf

2.4 Processing Paths

Path	Trigger	Stages traversed	Characteristics
Fast path	Known source with an approved, matching mapping	S1 → S5 → S2 → deterministic parser → S4 → S6 → output	No inference; lowest latency; the steady-state path for onboarded sources
Adaptive path	Unknown source, unknown format, or template mismatch	S1 → S5 → S2 → S3 → (S8 if low confidence) → S4 → S6 → output	Invoked during onboarding and on format drift; converges to the fast path once a mapping is approved
Degraded path	Mapping unavailable or inference failure	S1 → S5 → S2 → partial extraction → S4 (extensions only) → S6 → output	Event still emitted with preserved fields; flagged for review
Dead-letter path	Unrecoverable processing failure	S1 → S5 → dead-letter store	Raw event preserved; diagnostics recorded; never a silent drop

3. Component Design

3.1 S1 — Ingestion Gateway

3.1.1 Design

The gateway is a set of transport adapters sharing one internal contract. Each adapter converts its transport's delivery semantics into a canonical IngestRecord and pushes it onto the intake queue. Adapters hold no parsing logic whatsoever; this separation is what allows a new transport to be added without touching the pipeline.

```
IngestRecord {
  trace_id      : ULID          # allocated here, monotonic, sortable
  source_id     : string        # from the transport binding
  payload       : bytes         # untouched original
  byte_length   : int
  received_at   : timestamp     # ULPF clock, UTC, independent of source time
  transport     : enum{file,tcp,udp,http,broker}
  peer         : string|null    # remote address where applicable
  encoding_hint : string|null
}
```

3.1.2 Transport-specific handling

Transport	Delivery semantics	Design handling
File / directory watch	At-least-once; rotation and partial writes possible	Byte-offset checkpointing per inode; rotation detected by inode change; partial trailing line held until terminated
UDP Syslog	Fire-and-forget; datagram boundary is the event boundary	One datagram equals one event; oversized datagrams recorded as truncated with a flag; no reassembly attempted
TCP Syslog	Stream; framing ambiguity is the classic failure mode	Support both newline-delimited and RFC 6587 octet-counting; framing mode configurable per listener, auto-detected on first frames
HTTP / HTTPS	Request/response; batching common	Accept single object or array; acknowledge only after vault write completes, so retries are safe
Message broker	At-least-once with offset management	Commit offset only after vault write; duplicates suppressed downstream by trace_id idempotency

3.1.3 Backpressure

The intake queue is bounded. When it reaches its high-water mark the gateway stops acknowledging on acknowledgeable transports (HTTP, TCP, broker) and lets the client or broker retry. For UDP, where backpressure cannot be signalled, the gateway records a measured drop counter per source rather than pretending nothing happened. This distinction is stated explicitly because a system claiming zero loss on UDP would be making a claim the transport cannot support.

3.2 S2 — Format Detection Engine

3.2.1 Cascade design

Detection is a deterministic cascade, ordered by specificity. The first rule that matches with sufficient confidence wins; nothing further is evaluated. This ordering matters because CEF and LEEF are Syslog payloads and would otherwise be misclassified as generic Syslog.

Order	Check	Signal	Confidence
1	Source has a pinned format in its registration	Configuration	1.00
2	Payload begins with CEF: prefix	Literal grammar marker	0.99
3	Payload contains the LEEF: version marker	Literal grammar marker	0.99

Order	Check	Signal	Confidence
4	Payload parses as a JSON object or array	Strict parse success	0.98
5	Payload parses as well-formed XML	Strict parse success	0.97
6	Payload matches RFC 5424 priority and version header	Structural regex	0.95
7	Payload matches RFC 3164 priority and timestamp header	Structural regex	0.90
8	Consistent delimiter count across a sample window	Statistical consistency	0.70–0.90
9	Repeated key=value token structure above a density threshold	Token statistics	0.70–0.88
10	No rule matched	—	< 0.60 → route to discovery

3.2.2 Drift detection

For a source with an approved mapping, the detector also verifies that the event matches the approved template. A configurable proportion of consecutive non-matching events raises a drift signal, which surfaces in the console and re-opens the adaptive path for that source. Drift is treated as an operational event, not an error — the vendor changed something, and the operator needs to know.

3.3 S3 — Adaptive Parser Factory

This is the component that carries the project's novelty claim, so its internals are specified in detail.

3.3.1 Stage 1 — Template mining

Template mining uses Drain3, an online fixed-depth parse-tree algorithm that clusters log messages into templates incrementally without requiring the corpus in advance. This is the correct choice because it is streaming-capable, deterministic given a seed state, and persistable — all three properties matter for an air-gapped, restartable service.

Parameter	Value	Rationale
sim_th (similarity threshold)	0.4 initial, tuned per source	Lower values merge variants; higher values fragment templates. 0.4 is a stable starting point for security log corpora.
depth	4	Balances tree traversal cost against cluster precision for typical event token counts
max_children	100	Bounds fan-out for sources with high token cardinality in early positions
masking	IP, port, timestamp, hex, UUID, number, MAC	Pre-masking known value classes materially improves template convergence
persistence	File-backed snapshot	Restart resumes learned state; required for offline operation without re-learning

Honest scoping note: Drain3 is an existing open-source component and is not claimed as this project's innovation. Its role here is one stage of a larger workflow; the contribution is the orchestration around it.

3.3.2 Stage 2 — Field extraction and value typing

Variable positions identified by the template become candidate fields. Each is typed by an ordered pattern classifier, and a sample of observed values is retained to drive both typing confidence and reviewer display.

Inferred type	Recognition basis	Typical target
ipv4 / ipv6	Address grammar plus range validity	source.ip, destination.ip, device.ip
port	Integer within 0–65535, positionally adjacent to an address	source.port, destination.port
timestamp	Multi-format temporal parse with timezone inference	event.time_utc
mac	Six-octet hex with separator	source.mac
hostname / fqdn	Label grammar and TLD plausibility	device.hostname, destination.hostname
username	Identifier grammar with contextual key proximity	source.user
action verb	Membership in a controlled vocabulary of vendor action terms	event.action
severity	Numeric scale or named level vocabulary	event.severity
protocol	Membership in the IANA protocol name set	network.protocol
url / path	Scheme grammar or path grammar	url.original, file.path
numeric / boolean / text	Fallback classes	extensions namespace unless mapped

3.3.3 Stage 3 — Semantic mapping

Mapping proceeds in two tiers. The deterministic tier resolves field names present in a curated alias dictionary (for example `src_ip`, `sourceAddress`, `client_ip` and `originator` all resolving to `source.ip`). Only fields the deterministic tier cannot resolve reach the semantic tier.

The semantic tier embeds the candidate field name together with its template context and a sample of its values, embeds the core schema field descriptions once at startup, and ranks target fields by cosine similarity. The model is a compact sentence-transformer bundled in the image and executed on CPU. No inference request leaves the host.

3.3.4 Stage 4 — Composite confidence scoring

A single similarity number is not sufficient evidence to bind a field, because a plausible name can sit above an implausible value. The composite score combines four independent signals so that disagreement between them lowers confidence rather than being averaged away.

$$C(f \rightarrow t) = w1 * S_name + w2 * S_value + w3 * S_context + w4 * S_history$$

`S_name` name / alias / embedding similarity between source field and target
`S_value` agreement between the inferred value type and the target's expected type
`S_context` positional and co-occurrence consistency within the discovered template
`S_history` prior approved mappings for similar fields across onboarded sources

default weights `w1 = 0.35` `w2 = 0.30` `w3 = 0.20` `w4 = 0.15`

hard gate: if `S_value = 0` (type contradicts target), `C` is capped at `0.50` regardless of the other signals.

Band	Score	Action	Rationale
Auto-accept	$C \geq 0.90$	Mapping applied and recorded as auto-accepted	Multiple independent signals agree; error probability acceptable
Review	$0.65 \leq C < 0.90$	Mapping proposed; queued for human adjudication; not applied	Plausible but not established — exactly the case where silent application is dangerous
Extension-only	$C < 0.65$	Field preserved under extensions; flagged for review	Insufficient evidence to assert meaning; preservation is the safe default

Band	Score	Action	Rationale
Blocked	Type contradiction	Never auto-applied regardless of name similarity	Prevents a confidently-named but wrongly-typed binding

Thresholds are configurable per deployment. A site prioritising throughput may raise the auto-accept band; a site prioritising evidentiary rigour may lower it and accept a longer review queue. The design exposes this as policy rather than embedding one answer.

3.3.5 Stage 5 — Mapping registry and convergence

An approved mapping is written to the registry keyed by (source_id, template_id) with a version number. From the next event onward, that source takes the fast path. Onboarding cost is therefore paid once per source-template pair, not per event — which is the entire economic argument of the product.

3.4 S4 — Normalization Engine

3.4.1 Mapping application

The engine applies the resolved field bindings, runs value normalizers, and then performs the completeness check that enforces the lossless guarantee: the count of fields in the parsed representation must equal the count of fields in the normalized representation across core and extensions combined. A mismatch is a defect, not a warning, and fails the event to the dead-letter path.

3.4.2 Value normalizers

Normalizer	Behaviour	Original retained as
Timestamp	Parse multi-format, infer timezone, convert to UTC ISO-8601	event.time_original, event.timezone
Action	Map vendor terms (permit, allow, accept, pass → allow; deny, drop, block, reject → deny)	extensions.<ns>.original_action
Severity	Map vendor scale or named level to the common 0–10 scale	extensions.<ns>.original_severity
Protocol	Lowercase canonical IANA name; numeric protocol resolved to name	extensions.<ns>.original_protocol
Address	Validate and canonicalise IPv4/IPv6 representation	Original preserved in raw event
Case and whitespace	Trim and canonicalise identifiers without altering semantic content	Original preserved in raw event

3.4.3 Extension namespacing

Unmapped fields are placed under extensions.<namespace>.<field>, where the namespace is derived from the source's vendor and product labels. Namespacing is what prevents two vendors' identically named proprietary fields from colliding, and it is what allows the core schema to remain stable while the extension surface grows without limit.

3.5 S5 — Raw Event Vault

The vault is append-only object storage keyed by trace_id, with metadata in the relational store. Writes are ordered before any acknowledgement to the ingesting client, which is what makes the durability claim structural rather than best-effort.

Property	Design
Write semantics	Append-only; no update or in-place delete path exists in code
Integrity	SHA-256 digest computed at write; re-verified on every read; mismatch reported as an integrity failure, never silently repaired
Layout	Partitioned by source and ingestion date for retention efficiency and locality

Property	Design
Compression	Applied at the block level; digest is computed over uncompressed original bytes so verification is independent of storage encoding
Retrieval	By trace_id in constant time via a metadata index
Retention	Policy-driven expiry per source; every expiry recorded in the audit trail
Recoverability harness	Batch job that retrieves and digest-verifies every event in a corpus and reports the recovered/ingested ratio

3.6 S6 — Traceability Layer

Traceability is implemented at two granularities. Event-level binding is carried by the trace_id present on all three representations. Field-level provenance is a separate record per normalized field, written at normalization time — provenance is recorded, never reconstructed afterwards, because a reconstruction would be an inference rather than evidence.

```
ProvenanceRecord {
  trace_id      : ULID
  target_field  : string      # e.g. "source.ip"
  source_field  : string      # field name or positional index in the original
  source_value  : string      # value as it appeared before normalization
  transformation : string      # e.g. "direct", "tz_normalize", "action_vocab"
  mapping_id    : string
  mapping_version : int
  schema_version : string
  confidence    : float
  decision      : enum{auto_accepted, human_approved, extension_only}
}
```

Reverse lookup resolves a normalized event to its raw record by trace_id. Forward lookup resolves a raw record to the normalized events derived from it, which is a one-to-many relationship when an event is reprocessed under a newer schema version.

3.7 S7 — Schema Registry

Concern	Design decision
Versioning scheme	Semantic: major.minor. Additive changes increment minor; removals or type changes require a major version
Immutability	A published version is content-addressed; any edit produces a new version
Compatibility rule	Within a major version, every prior valid event remains valid — enforced by a compatibility check at publish time
Profiles	OCSF-aligned core is authoritative; ECS is a rendering profile over the same event, not a separate model
Extension governance	Namespaces are reserved per vendor/product; namespace collision with core field names is rejected at registration
Pinning	Each source pins a schema version, so upgrading the registry does not implicitly change any existing source's output
Portability	Schema versions and mapping packages export as signed files for transfer between air-gapped sites

3.8 S8 — Review Console and API Layer

The console exists because the confidence bands in §3.3.4 are only meaningful if something acts on them. Its central screen is the mapping review, which shows the source field, its sample values, the inferred type, the proposed target, the four constituent confidence signals and their weighted result. Showing the constituent signals rather than only the total is a deliberate usability requirement: a reviewer who can see that name similarity was high but value agreement was zero can adjudicate in seconds.

The side-by-side inspector renders the raw event and the normalized event with per-field linkage drawn between them, which is also the artefact used for the live demonstration. Every reviewer action writes an audit entry and creates a new mapping version; nothing is edited in place.

4. Data Design

4.1 Persistence Allocation

Different data classes have genuinely different access patterns, and each store is chosen for one of them. The prototype does not deploy every store; components are included only when the workload justifies them.

Store	Holds	Why this store	In prototype
Object storage / filesystem	Raw Event Vault payloads	Append-only, cheap, large-object friendly, trivially backed up	Yes
PostgreSQL	Sources, templates, mappings, schema versions, provenance, audit, dead letters	Durable relational metadata with transactional integrity and versioning constraints	Yes
OpenSearch	Normalized events for exploration and query	Inverted-index search over semi-structured events with extension fields	Yes
Redis	Template cache, mapping cache, dedup keys, rate counters	Low-latency lookup on the hot path; loss is tolerable because it is a cache	Optional
Kafka	Partitioned event stream between gateway and workers	Buffering, decoupling and partitioned parallelism at high volume	Optional

On the “why three databases” question: these are architectural options, not mandatory components. PostgreSQL provides durable metadata, Redis provides low-latency hot-path state, and Kafka is introduced only when high-volume streaming and horizontal worker scaling are required. The prototype deploys only what its measured workload justifies, and the deployment profile in §7.2 makes that explicit.

4.2 Core Relational Schema

```

sources(source_id PK, name, transport, vendor, product, namespace,
        schema_pin, format_pin, status, created_at)

templates(template_id PK, source_id FK, pattern, variable_positions JSONB,
          first_seen, last_seen, occurrence_count, status)

mappings(mapping_id PK, source_id FK, template_id FK, version,
          field_bindings JSONB, status, confidence_summary,
          approved_by, approved_at, superseded_by)

schema_versions(schema_version PK, published_at, field_definitions JSONB,
                compatibility_class, checksum)

raw_index(trace_id PK, source_id FK, received_at, transport, peer,
          byte_length, digest, storage_uri, expires_at)

provenance(trace_id FK, target_field, source_field, source_value,
           transformation, mapping_id FK, mapping_version,
           schema_version, confidence, decision,
           PRIMARY KEY(trace_id, target_field))

audit(audit_id PK, actor, action, subject_type, subject_id,
      before JSONB, after JSONB, occurred_at)      -- append-only

dead_letters(trace_id PK, stage, error_class, diagnostic,
             raw_reference, occurred_at)

```

4.3 Indexing Strategy

Index	On	Serves
raw_index(trace_id)	Primary key	Constant-time raw event retrieval for trace-back
raw_index(source_id, received_at)	Composite	Retention expiry and per-source volume reporting

Index	On	Serves
provenance(trace_id, target_field)	Composite primary key	Field-level provenance lookup in a single seek
mappings(source_id, template_id, version DESC)	Composite	Resolving the current mapping on the fast path
templates(source_id, pattern hash)	Composite	Template match on ingest without a full scan
OpenSearch: time + source + event.category	Search index	Analyst-facing filtering and dashboards

4.4 Identifier Design

Trace identifiers are ULIDs rather than random UUIDs. The choice is deliberate: ULIDs are lexicographically sortable by generation time, which makes time-ordered scans over the vault index efficient and makes partition pruning by identifier prefix possible without a separate timestamp lookup. They remain globally unique across concurrent workers without coordination, which matters for the horizontally scaled configuration.

5. API Design

5.1 Conventions

- All endpoints are versioned under `/api/v1` and served over TLS.
- Authentication is bearer-token based; every endpoint requires it. There is no anonymous read path to event or provenance data.
- Errors follow a single problem-detail structure carrying a machine-readable code, a human-readable message, the pipeline stage where applicable, and the `trace_id` where applicable.
- List endpoints are cursor-paginated. Offset pagination is not offered, because it is unreliable over a continuously appended dataset.
- An OpenAPI document is served by the system itself, so an air-gapped operator never needs external documentation.

5.2 Endpoint Catalogue

Method	Path	Purpose	Role
POST	<code>/api/v1/ingest</code>	Submit one event or a batch for ingestion	ingest
POST	<code>/api/v1/sources</code>	Register a new source with transport and labels	admin
GET	<code>/api/v1/sources</code>	List registered sources with status and volume	viewer
PATCH	<code>/api/v1/sources/{id}</code>	Update source configuration, pins or rate limits	admin
POST	<code>/api/v1/onboarding/sample</code>	Submit a sample for structure discovery	approver
GET	<code>/api/v1/onboarding/{job}/proposal</code>	Retrieve discovered template and proposed mapping with confidence	approver
POST	<code>/api/v1/mappings/{id}/approve</code>	Approve, correct or reject a proposed mapping	approver
GET	<code>/api/v1/mappings</code>	List mappings with version, status and approval metadata	viewer
GET	<code>/api/v1/review/queue</code>	List proposals awaiting human adjudication	approver
GET	<code>/api/v1/events</code>	Query normalized events with filters and cursor pagination	viewer
GET	<code>/api/v1/events/{trace_id}</code>	Retrieve one normalized event	viewer
GET	<code>/api/v1/events/{trace_id}/raw</code>	Retrieve the preserved original event with digest verification	viewer
GET	<code>/api/v1/events/{trace_id}/provenance</code>	Retrieve the full derivation chain for the event	viewer
GET	<code>/api/v1/events/{trace_id}/provenance/{field}</code>	Retrieve derivation for one normalized field	viewer
GET	<code>/api/v1/schemas</code>	List schema versions	viewer
GET	<code>/api/v1/schemas/{version}</code>	Retrieve a schema version definition	viewer
POST	<code>/api/v1/export</code>	Trigger a JSON, NDJSON or Parquet export for a query	viewer
GET	<code>/api/v1/deadletters</code>	List failed events with diagnostics	admin

Method	Path	Purpose	Role
GET	/api/v1/metrics	Operational metrics for monitoring	admin
GET	/api/v1/health	Liveness and readiness	public

5.3 Representative Response — Provenance

GET /api/v1/events/ULPF-8F91-4C2A-B7D0/provenance

```
{
  "trace_id": "ULPF-8F91-4C2A-B7D0",
  "schema_version": "ulpf-core-1.2",
  "mapping": { "id": "map-fw", "version": 3,
    "approved_by": "m.iyer", "approved_at": "2026-08-20T09:12:44Z" },
  "raw_ref": { "digest": "sha256:9c1e...", "verified": true,
    "byte_length": 58, "received_at": "2026-08-22T13:11:02Z" },
  "fields": [
    { "target": "source.ip", "source_field": "pos_3",
      "source_value": "10.2.1.5", "transformation": "direct",
      "confidence": 0.96, "decision": "auto_accepted" },
    { "target": "destination.port", "source_field": "pos_5b",
      "source_value": "22", "transformation": "split_proto_port",
      "confidence": 0.91, "decision": "auto_accepted" },
    { "target": "event.action", "source_field": "pos_6",
      "source_value": "DENY", "transformation": "action_vocab",
      "confidence": 0.88, "decision": "human_approved" },
    { "target": "extensions.fwx.RID", "source_field": "RID",
      "source_value": "82931", "transformation": "preserve",
      "confidence": 0.41, "decision": "extension_only" }
  ]
}
```

The last entry is the important one. A field the system could not confidently interpret is neither guessed at nor discarded — it is preserved, marked, and left available to a human or a downstream consumer who knows what RID means.

6. Technology Stack

6.1 Selections and Rationale

Layer	Technology	Rationale	Alternative considered
Core processing	Python 3.11	Strongest ecosystem for log parsing, template mining and embedding inference; team fluency	Go — faster runtime, far weaker ML tooling
API layer	FastAPI	Async, typed request models, OpenAPI generated from code rather than maintained separately	Flask — less native async and schema tooling
Template mining	Drain3	Online, streaming, persistable, proven on log corpora, permissively licensed	Spell, LogMine — less mature tooling
Semantic mapping	Sentence-Transformers (compact CPU model)	Runs offline on CPU with acceptable latency; weights bundled at build time	LLM API — disqualified by the air-gap constraint
Metadata store	PostgreSQL 15	Transactional integrity for versioning constraints; JSONB for flexible bindings	SQLite — insufficient for concurrent workers
Search / analytics	OpenSearch	Open licence, mature index for semi-structured events, self-hostable	Elasticsearch — licence complexity for redistribution
Cache / hot state	Redis	Sub-millisecond template and mapping lookup on the fast path	In-process cache — not shared across workers
Stream buffer	Kafka (optional)	Partitioned parallelism and durable buffering when volume warrants	Redis Streams — adequate at lower volume
Frontend	React + Tailwind CSS	Component model suits the mapping-review interaction; utility CSS keeps the bundle small	Server-rendered templates — weaker interactivity
Packaging	Docker + Docker Compose	Single-command deployment; offline image bundle satisfies the air-gap requirement	Kubernetes — disproportionate for the prototype

6.2 Offline Dependency Policy

- All Python dependencies are pinned by hash and vendored into the build image; no package index is contacted at runtime.
- Embedding model weights are baked into the image at build time. A runtime download path does not exist in the code.
- Container images are exported as a tar bundle that can be transferred on physical media and loaded on the target host.
- Licences of all bundled components are permissive and compatible with redistribution in an offline bundle; a licence manifest ships with the release.

7. Deployment Architecture

7.1 Service Topology

```
docker compose up

ulpf-gateway    ingestion adapters, trace_id allocation, vault write
ulpf-worker    detection, discovery, mapping, normalization (scalable, N replicas)
ulpf-api       REST API, provenance service, export service
ulpf-console   React UI served as static assets behind the API
postgres      metadata, mappings, provenance, audit
opensearch     normalized event index
redis          hot-path cache (profile: standard)
kafka          partitioned event buffer (profile: scale)

volumes: vault-data (append-only) pg-data os-data drain3-state
```

7.2 Deployment Profiles

Profile	Services	Target scenario	Hardware
minimal	gateway, worker, api, console, postgres	Demonstration, single-source pilot, constrained hardware	4 vCPU / 8 GB
standard	minimal + opensearch + redis	Normal enterprise deployment with analyst-facing search	8 vCPU / 16 GB
scale	standard + kafka + N workers	High-volume ingestion requiring partitioned parallelism	Multi-node

7.3 Air-Gapped Installation

Step	Action	Verification
1	Build the release bundle on a connected build host; export images as a tar archive with the licence manifest	Bundle checksum recorded
2	Transfer the bundle to the isolated network on approved physical media	Checksum re-verified on the target host
3	Load images into the local container runtime	Image digests match the manifest
4	Provide site configuration: sources, transports, thresholds, TLS certificates	Configuration schema validation passes
5	Start the stack with a single Compose command	All health endpoints report ready
6	Run the air-gap verification suite with an egress monitor active	Zero outbound connections observed during a full processing run

7.4 Scalability Design

Horizontal scale is achieved by partitioning the event stream by `source_id` and running stateless workers against the partitions. Partitioning by source rather than round-robin preserves per-source ordering, which matters because template mining is stateful with respect to a source's event sequence.

Two honesty constraints govern how this is presented. First, the prototype reports measured single-node throughput on stated hardware with a stated workload, not an extrapolated figure. Second, the architecture describes the path to higher scale without claiming that path has been exercised at that scale. The claim made is that the design is horizontally scalable and that the measured baseline is X events per second; the claim not made is a specific unmeasured billion-event figure.

Scaling dimension	Mechanism	Constraint
Ingestion	Additional gateway replicas behind a load balancer per transport	UDP requires per-listener affinity

Scaling dimension	Mechanism	Constraint
Processing	Additional stateless workers consuming partitions	Ordering guaranteed within a source, not across sources
Vault writes	Partitioned object layout by source and date	Write throughput bounded by underlying storage
Query	OpenSearch shard scaling	Index size bounded by retention policy
Template state	Per-source Drain3 state; affinity of a source to a worker where possible	Cross-worker state sharing requires the shared cache

8. Security Design

8.1 Threat Model

ULPF ingests untrusted input by definition — log content is written by devices that may themselves be compromised. The threat model is dominated by that fact.

Threat	Vector	Control
Log injection	Crafted log content designed to be misinterpreted as structure or to inject into downstream queries	Content is never evaluated as code; all parsing is data-driven; output encoding on every sink; no string-concatenated queries
Resource exhaustion	Oversized events, pathological delimiter density, template explosion	Per-event size caps, bounded template cardinality per source, bounded intake queue, per-source rate limits
Mapping poisoning	A source deliberately emitting field names designed to induce a wrong mapping	Type gate on confidence scoring; human review band; the raw event remains authoritative
Unauthorised evidence access	Retrieval of raw events containing sensitive content	Authentication on every endpoint; role separation; raw access is a distinct privilege from normalized access
Data exfiltration	Any path sending log content outside the perimeter	No outbound call exists on the processing path; verified by egress monitoring in the air-gap test
Tampering with evidence	Modification of a stored raw event	Append-only vault with no update path in code; digest verified on every read
Audit repudiation	Denial that a mapping was approved by a given actor	Append-only audit trail with actor identity and timestamp on every governance action

8.2 Security Controls

Domain	Control
Transport	TLS on all externally exposed HTTP interfaces; configurable certificates; TLS optional for internal service-to-service traffic within the compose network
Authentication	Bearer tokens for API clients; session authentication for console users; no anonymous access to event or provenance data
Authorisation	Three roles — viewer (read events and provenance), approver (viewer plus mapping decisions), administrator (full configuration and raw access)
Input handling	Size caps, encoding validation, parser timeouts, and treatment of all content as data rather than executable structure
Secrets	Supplied via environment or mounted secret files; never embedded in images or committed configuration
Privacy	Optional field-level masking on the normalized event for configured sensitive fields, with the raw event protected by the separate raw-access privilege
Auditing	Append-only audit of every mapping approval, correction, schema publication, configuration change and retention expiry
Supply chain	Pinned and hash-verified dependencies; licence manifest; reproducible image build

9. Performance Engineering & Observability

9.1 Performance Budgets

Stage	Budget (fast path, p95)	Design note
Ingestion + trace allocation	< 5 ms	No parsing performed; ULID generation is local
Vault write + digest	< 20 ms	Sequential append; digest computed in the same pass over the buffer
Format detection	< 5 ms	Cascade short-circuits on the first confident match
Deterministic parse	< 15 ms	Compiled patterns, no inference
Normalization + provenance	< 25 ms	Bounded by field count, not by event size
Total fast path	< 100 ms	Aggregate target stated in NFR-P3
Adaptive discovery (interactive sample)	< 60 s	Includes template convergence and embedding inference

9.2 Optimisation Strategy

- Keep inference off the hot path entirely. Once a mapping is approved, no embedding computation occurs for that source — this is the single largest performance decision in the design.
- Cache compiled parsers, template state and resolved mappings in the shared cache, keyed by source and template.
- Compute the digest in the same pass that writes the payload, avoiding a second traversal of the buffer.
- Batch writes to the metadata store and the search index rather than issuing one round trip per event.
- Bound template cardinality per source so that a pathological source cannot degrade global memory.

9.3 Observability

Signal	Metrics exposed
Throughput	Events ingested, parsed, normalized and delivered per second, by source and transport
Latency	Per-stage duration histograms; end-to-end p50, p95 and p99
Path distribution	Proportion of events on the fast, adaptive, degraded and dead-letter paths
Confidence health	Distribution of mapping confidence scores; auto-accept versus review versus extension-only ratios
Review load	Review queue depth, age of oldest item, decisions per reviewer
Preservation	Vault write success rate, digest verification failures, recoverability ratio
Drift	Template mismatch rate per source; drift signals raised
Errors	Dead-letter rate by stage and error class; parser timeouts; dropped UDP datagrams by source
Resources	CPU, memory, queue depth, storage utilisation and retention headroom

10. Test & Benchmark Strategy

10.1 Test Levels

Level	Coverage	Tooling
Unit	Parsers, type inference, value normalizers, confidence computation, schema validation	pytest with coverage reporting
Integration	Transport to normalized output for each format family, including the degraded and dead-letter paths	Containerised pipeline harness
Corpus	The full universality matrix defined in the SRS — formats, sources, structures, schema conditions, transports, adversarial inputs	Curated corpus with ground-truth labels
Accuracy	Field mapping accuracy against a manually labelled ground-truth field set	Comparison harness producing a per-field confusion report
Preservation	Byte-identical recovery and digest verification across the entire corpus	Recoverability audit job
Performance	Throughput and latency on stated reference hardware and workload	Load generator with fixed event mix
Security	Injection resistance, resource-exhaustion resistance, authorisation enforcement	Adversarial input suite and authorisation matrix tests
Air-gap	Installation and full operation with no outbound route	Egress monitor during a complete processing run

10.2 Benchmark Methodology

Every published figure must be reproducible, so each benchmark result is reported together with the conditions that produced it. A number without its conditions is treated as unpublishable.

Element	Specification
Hardware	Stated explicitly: vCPU count, memory, storage class, container runtime version
Workload	Stated event mix by format and size distribution; stated ratio of fast-path to adaptive-path events
Warm-up	Measurements taken after cache and template state have reached steady state; warm-up period stated
Duration	Sustained run of defined length; peak and sustained figures reported separately
Repetition	Minimum three runs; median reported with observed variance
Reporting	Measured figures replace target figures only after this procedure; unmeasured values remain labelled as targets

10.3 Measurement Definitions

Metric	Formula / definition
New-source onboarding time	Timestamp of first validated normalized event minus timestamp of first raw event received from that source
Source-specific parser code	Lines of bespoke code authored to onboard the source; target is zero
Field mapping accuracy	Correctly mapped fields ÷ total fields in the labelled ground-truth set, evaluated per source and aggregated
Raw-event recoverability	Events recovered byte-identically with digest verification passing ÷ events ingested

Metric	Formula / definition
Unknown-field preservation	Unmapped source fields present in the normalized event ÷ unmapped source fields received
Coverage	Successfully processed test categories ÷ total categories in the universality matrix
Throughput	Events successfully normalized per second, sustained, under the stated workload
Normalization latency	Elapsed time from ingestion acknowledgement to standardised event emission, reported as p50 and p95

11. Technical Risks & Design Responses

ID	Technical risk	Design response	Residual
TR-1	Template mining fails to converge for highly variable free-text sources	Pre-masking of known value classes; per-source similarity tuning; fall back to delimiter and grammar heuristics; preserve as extensions rather than failing	Some free-text sources will need a manual hint; accepted and documented
TR-2	Semantic mapping proposes a plausible but wrong target	Deterministic tier takes precedence; type-contradiction gate caps confidence; review band; raw event remains authoritative	Residual risk is bounded by the review threshold, which is a policy choice
TR-3	Embedding inference latency dominates processing cost	Inference occurs only during onboarding, never on the fast path; embeddings cached per field-name string	Onboarding latency remains inference-bound; acceptable within the 60-second budget
TR-4	Template state growth is unbounded for a pathological source	Per-source template cardinality cap with eviction; alert on cap approach	A source exceeding the cap gets degraded discovery quality, surfaced to operators
TR-5	Vault storage grows faster than provisioned capacity	Per-source retention policy, compression, storage headroom metric with alerting	Retention shortening is an operational decision, made visible rather than automatic
TR-6	UDP transport loses events before ULPF ever sees them	Measured drop counters per source; TCP recommended in deployment guidance	Cannot be eliminated — stated as a transport property rather than concealed
TR-7	Schema evolution invalidates historical normalized events	Immutable versions, additive-only within a major version, per-source pinning, reprocessing from the vault	Major version transitions require deliberate migration
TR-8	Search index becomes the throughput bottleneck	Batched indexing, index decoupled from the normalization path, index failure degrades search but not preservation	Search may lag the pipeline under peak load

12. Appendix

12.1 Worked Example — Onboarding an Unseen Source

The sequence below is the technical narrative behind the live demonstration. Each step names the component and the artefact it produces.

```

INPUT  (never seen before, arrives over UDP Syslog)
2026-08-22 18:41 FW-X 10.2.1.5 -> 10.5.4.8 TCP/22 DENY RID=82931

S1 Ingestion
   trace_id = ULPF-8F91-4C2A-B7D0 ; received_at = 2026-08-22T13:11:02Z

S5 Vault write (BEFORE any parsing)
   digest = sha256:9c1e...   byte_length = 58   uri = vault://fwx/2026-08-22/...

S2 Format detection
   CEF? no. LEEF? no. JSON? no. XML? no. RFC5424? no. RFC3164? partial.
   delimiter consistency 0.55 -> below 0.60 -> route to discovery

S3 Template mining (Drain3, masking applied)
   <TIME> <WORD> <IP> -> <IP> <WORD>/<NUM> <WORD> RID=<NUM>

S3 Field extraction and typing
pos_1 2026-08-22 18:41 timestamp
pos_2 FW-X hostname
pos_3 10.2.1.5 ipv4
pos_4 10.5.4.8 ipv4
pos_5 TCP/22 protocol + port (composite)
pos_6 DENY action verb
RID 82931 numeric, unknown semantics

S3 Mapping proposals with composite confidence
pos_1 -> event.time_utc C = 0.97 auto
pos_2 -> device.hostname C = 0.93 auto
pos_3 -> source.ip C = 0.96 auto (arrow direction gives context)
pos_4 -> destination.ip C = 0.96 auto
pos_5 -> network.protocol C = 0.94 auto
      -> destination.port C = 0.91 auto
pos_6 -> event.action C = 0.88 REVIEW (0.65 <= C < 0.90)
RID -> no target C = 0.41 EXTENSION-ONLY

S8 Human review
Reviewer confirms pos_6 -> event.action. Mapping map-fwv v3 approved.
RID left as extensions.fwx.RID – preserved, not guessed.

S4 Normalization -> unified event (see SRS §5.3)
S6 Provenance -> 8 field records written

```

```

RESULT
New parser code written      : 0 lines
Fields preserved             : 8 of 8
Raw event recoverable        : yes, digest verified
Subsequent events from FW-X  : fast path, no inference

```

12.2 Configuration Sketch

```

sources:
- id: fw-x-dmz
  name: "Perimeter Firewall (DMZ)"
  vendor: fwx
  product: fw-series
  namespace: fwx
  transport:
    type: udp
    port: 5514
  schema_pin: ulpf-core-1.2
  rate_limit_eps: 20000

thresholds:
  detection_min_confidence: 0.60
  mapping_auto_accept: 0.90

```

```

mapping_review_floor:    0.65

discovery:
  drain3:
    sim_th: 0.4
    depth: 4
    max_children: 100
    masking: [ip, port, timestamp, hex, uuid, number, mac]
    max_templates_per_source: 500

preservation:
  vault_retention_days: 90
  digest_algorithm: sha256

outputs:
  - type: opensearch
    index: ulpf-events
  - type: parquet
    path: /data/export
  - type: syslog_forward
    host: siem.internal
    port: 514

```

12.3 Component Traceability

Component	SRS requirements implemented	PRD features
S1 Ingestion Gateway	FR-1.1 – FR-1.10, IR-3.1 – IR-3.5	F1.1 – F1.6
S2 Format Detection	FR-2.1 – FR-2.7	F2.1 – F2.4
S3 Adaptive Parser Factory	FR-3.1 – FR-3.11	F3.1 – F3.7
S4 Normalization Engine	FR-4.1 – FR-4.11	F4.1 – F4.6
S5 Raw Event Vault	FR-5.1 – FR-5.9, DR-I1	F5.1 – F5.5
S6 Traceability Layer	FR-6.1 – FR-6.9, DR-7	F6.1 – F6.5
S7 Schema Registry	FR-7.1 – FR-7.8	F7.1 – F7.5
S8 Console & API	FR-8.1 – FR-8.9, FR-9.1 – FR-9.7, IR-1.1 – IR-1.7	F8.1 – F8.6, F9.1 – F9.5
Deployment	FR-10.1 – FR-10.8, NFR-M4, NFR-M5	F10.1 – F10.6

12.4 Positioning Statement for Technical Review

Drain3, OCSF and ECS are building blocks and are not claimed as this project's innovation. Drain3 performs template mining; OCSF and ECS provide established schema foundations. The technical contribution specified in this document is the orchestration layer that connects adaptive source discovery, deterministic-first semantic mapping, composite confidence validation, write-before-transform preservation, field-level provenance, versioned schema evolution and fully offline deployment into a single preprocessing workflow — together with the measurement framework that determines whether that workflow actually reduces onboarding effort without sacrificing evidence.